

RCS + VRF Subnet — Pipeline Architecture

A seven-leg architecture for provably-quantum, on-chain-verifiable randomness

Personal working notes

1. Document purpose

This document elaborates on the architecture of the proposed RCS + VRF subnet, synthesizing the certification mechanism from Liu et al. 2025 (random circuit sampling with cross-entropy benchmarking) with the traceability machinery from Kavuri et al. 2025 (the Twine protocol of distributed intertwined hash chains). It's a working specification — detailed enough to show the architecture is coherent and the design choices are defensible, while flagging the open questions that benchmarking and team conversations will resolve.

The audience is anyone wanting to understand how the subnet would actually work end-to-end: what each component does, what attack it defends against, what comes from which paper, and what's still undecided.

2. One-page summary

Goal. A Bittensor subnet that produces *certified-random bits* — output that is (a) genuinely random, originating from quantum measurement, (b) classically verifiable, so any participant can confirm the bits weren't faked, and (c) on-chain auditable, so the entire generation process is tamper-evident.

Why this is a proof-of-quantumness service. The certification rests on a complexity-theoretic argument: a quantum computer can sample from the output distribution of a random circuit faster than any classical attacker can fake samples that pass the same statistical test. A miner without quantum hardware therefore cannot produce valid output — so participation in the subnet *is* evidence of quantum capability. This is what positions the subnet as a proof-of-quantum-work mechanism that incentivizes major quantum providers (IonQ, Quantinuum, IBM, Google) to compete.

The pipeline in one line. VRF contributions → parameter precommitment → combined seed → deterministic circuit → multi-host quantum sampling → XEB verification with timing check → extraction with independent seed → Twine-anchored on-chain publication.

Seven legs, two complementary security guarantees.

- **VRF certifies the input.** It guarantees the seed used to generate the quantum circuit was produced honestly — no participant could have biased it, ground for favorable outputs, or kept it hidden from others.

- **XEB certifies the output.** It guarantees the bitstrings returned by miners really came from running the circuit on quantum hardware, not faked classically. XEB is about integrity of the response; it guarantees the work was done quantumly.
- These two sit at different layers and are complementary, not alternatives. Both are required.
- Twine hash chains wrap everything, making the entire pipeline cryptographically auditable end-to-end.

Where the ingredients come from.

Leg	Source
1. VRF-based seed generation	Chainlink VRF / RANDAO patterns
2. Protocol parameter precommitment	Kavuri et al. (PEF precommit pattern)
3. Deterministic circuit specification	Liu et al. (circuit family + PRG expansion)
4. Multi-host quantum sampling	Liu et al. + Kavuri (three-party generalization)
5. Classical XEB verification	Liu et al. (XEB + timing check)
6. Randomness extraction	Liu et al. (Toeplitz/Cryptomite) or Kavuri (TMPS Trevisan)
7. Twine-style hash-chain anchoring	Kavuri et al.

3. Leg 1 — VRF-based seed generation

Role. Produce an unbiased, publicly verifiable seed that nobody could have ground for favorable outputs.

Inputs. A shared public input x that no participant controls. Typically: previous round’s output, the current block height, and a public nonce, concatenated.

Process. Each of n classical participants on the subnet holds a VRF keypair (sk_i, pk_i) with public keys pk_i registered on-chain. On the shared input x , each participant computes:

$$(y_i, \pi_i) = \text{VRF}_{sk_i}(x)$$

where y_i is the VRF output (looks uniformly random to anyone without sk_i) and π_i is a non-interactive zero-knowledge proof that y_i is the unique correct output for input x under public key pk_i . Each participant publishes (y_i, π_i) on-chain.

Anyone with pk_i can run the verification algorithm and confirm (y_i, π_i) is legitimate without learning anything about sk_i . Invalid contributions are discarded.

The combined seed is then computed deterministically:

$$\text{seed} = H(y_1 \| y_2 \| \dots \| y_k)$$

across all valid contributions, using a cryptographic hash H (SHA-3 or BLAKE3). The seed is ~ 256 bits.

Outputs. A 256-bit seed, plus the set of (y_i, π_i) tuples preserved for audit.

Properties enforced.

- **Unbiasability.** No single participant can choose the seed: each VRF output is the unique correct value given their secret key and the public input, so they can't grind for favorable outputs.
- **Public verifiability.** Anyone can re-run the verification on each (y_i, π_i) and re-hash to confirm the seed is the correct deterministic function of public inputs.
- **Unpredictability before publication.** Until participants publish their y_i , nobody (including other participants) can predict the seed — it depends on every participant's secret key.

Why VRF instead of RANDAO. RANDAO is multi-party commit-reveal: participants commit hashes of secrets, then reveal. The known weakness is the last-revealer attack — whoever reveals last can see all other contributions and choose whether to publish their own (biasing the result). VRFs eliminate this: there's no commit-reveal asymmetry, just “produce the unique VRF output or be marked as abstaining.” A participant who doesn't like their contribution cannot pretend their machine crashed and avoid releasing their value while keeping the option to release.

Open questions. The team's preferred VRF construction (ECVRF? threshold BLS?) needs to be confirmed with Medi. If many independent participants are expected, threshold-VRF or aggregate VRF schemes could distribute trust further.

4. Leg 2 — Protocol parameter precommitment

Role. Bind the protocol's verification parameters *before* the seed is finalized, eliminating adaptive-choice attacks where parameters could be tuned after seeing the seed.

Inputs. The current protocol epoch number, plus protocol-defined defaults for parameters that may be updated.

Process. On the subnet's parameter hash chain, the following are publicly committed for the round before the VRF contributions in Leg 1 are finalized:

- Circuit family description (gate set, topology, layer pattern — e.g., “Liu et al. edge-coloured graph $SU(2)+U_{ZZ}$ at $n = 32$, depth $d = 10$ ”).
- The PRG used to expand seed $\rightarrow$ circuit (e.g., SHAKE-256).
- Number of circuits per round M .
- Number of shots per circuit.
- XEB acceptance threshold χ .
- Timing threshold $t_{\text{threshold}}$.
- Size of verification subset m chosen by validators.
- Randomness extractor used and its parameters.
- Soundness target ε_{sou} .

- Independent-seed source for the extractor (e.g., next drand pulse).

Outputs. A signed, hash-chained parameter commitment block referencing the round number.

Properties enforced.

- **No adaptive parameter choice.** Validators can't tune χ down after seeing bad samples, or up after seeing good ones.
- **Symmetry of expectations.** All participants (miners, validators, on-chain consumers, external auditors) work from the same parameter set without trust.
- **Replayability.** Any external verifier can read the parameter commitment and re-run the round end-to-end.

Why this leg matters. This is the most underrated lesson from Kavuri. Their PEFs are recalibrated daily but committed before each round's Bell data is taken. Without this pattern, a malicious validator could observe bad samples and selectively loosen χ to accept them, or observe good samples and selectively tighten χ to reject (and thus deny rewards). Precommitment makes the parameter choice non-adversarial.

Open questions. How often are parameters updated (per round? per epoch? per protocol upgrade?). Whether the parameter commitment is on the main chain or a sidechain. The governance process for parameter changes.

5. Leg 3 — Deterministic circuit specification

Role. Transform the 256-bit seed into a complete, reproducible quantum circuit description.

Inputs. The seed from Leg 1, plus the circuit family and PRG specification from Leg 2.

Process. The seed feeds a deterministic pseudorandom generator (SHAKE-256 in PRG mode, ChaCha20, or similar), which produces a long pseudorandom byte stream. The stream is consumed by the circuit generation algorithm.

Following Liu et al.'s recipe:

1. Sample a random graph G on n qubits — typically an edge-coloured random graph with all-to-all connectivity, where the colouring determines the order of two-qubit-gate application.
2. For each of d layers:
 - Sample n independent random $SU(2)$ single-qubit gates (each specified by three Euler angles drawn from the PRG stream).
 - Apply fixed U_{ZZ} two-qubit entangling gates along the edges of G , in the order determined by the edge colouring (gates of the same colour applied in parallel, different colours sequenced).

The result is a complete circuit description C : a list of (gate, qubits, parameters) tuples.

Outputs. A circuit specification C — typically serialized as a JSON or OpenQASM document — fully determined by the seed and the parameter commitment.

Properties enforced.

- **Determinism.** Same seed + same parameters $\rightarrow$ same circuit. Always.
- **Public reproducibility.** Validators, external auditors, and any participant can independently regenerate C from the public seed and parameter commitment.
- **Pseudorandomness over the circuit family.** If the seed is uniformly random (Leg 1 ensures this), the circuit is statistically indistinguishable from a uniformly drawn circuit in the family, up to small computational distinguishing advantage.
- **Cannot be precomputed.** Since the seed is unknown until Leg 1 finalizes, miners can’t prepare samples in advance. The seed-to-circuit-to-miner path must complete within a window short enough that the miner’s quantum hardware is the only realistic compute resource available — this addresses Colton’s fourth requirement (“cannot be pre-prepared”).

Why this circuit family. The Liu et al. choice (SU(2) random rotations + U_{ZZ} entanglers on edge-coloured graphs) achieves three things together: (1) the SU(2) rotations provide local randomness, (2) the edge-coloured graph provides rapid mixing across qubits (faster than 2D nearest-neighbor), and (3) the structure is an approximate 2-design at modest depth — meaning XEB-relevant moments match Haar-random behavior to good approximation. Sycamore-style brickwork and IBM heavy-hex circuits are alternative choices; the right pick depends on which platforms our miners use.

Open questions. Whether to use Liu et al.’s exact recipe or a variant. Whether to fix one circuit family for the whole protocol or allow per-platform variants (e.g., Quantinuum miners use all-to-all, IBM miners use heavy-hex). The latter would need separate validator simulations per platform.

6. Leg 4 — Multi-host quantum sampling

Role. Execute the circuit on real quantum hardware and return measurement samples, fast enough that classical simulation by an attacker can’t keep up.

Inputs. Circuit specification C , distributed to one or more quantum miners.

Process. Each miner takes C and submits it to their quantum hardware (IonQ, Quantinuum H-series, IBM Heron, Google Willow, etc.). The hardware:

1. Initializes n qubits in the $|0\rangle^{\otimes n}$ state.
2. Applies the gates of C in order.
3. Measures all n qubits in the computational basis.
4. Returns a bitstring $x \in \{0, 1\}^n$.

This is one shot. The miner repeats this M times (or shots-per-circuit-per-miner times if multiple circuits are batched). Each shot is an independent run; results are independent samples from the circuit’s output distribution $p(x) = |\langle x|C|0\rangle|^2$.

The miner must return all M samples within the protocol-specified time window $t_{\text{threshold}}$. If samples arrive late, they are rejected even if otherwise valid.

The multi-host generalization (from Kavuri). Rather than a single quantum miner per round, the subnet supports multiple miners running the same circuit in parallel (or running related circuits whose outputs combine). This is the structural analog of Kavuri’s three-party split (NIST/CU/drand), lifted to the quantum sampling layer. The benefits:

- **No single-vendor trust.** If IBM’s hardware were silently compromised, IonQ’s and Quantinuum’s still produce honest output.
- **Cross-validation.** Different miners’ samples on the same circuit should produce statistically similar XEB scores; large divergences flag a misbehaving miner.
- **Economic competition.** Miners with higher fidelity earn proportionally more, incentivizing fidelity improvements.
- **Resilience to vendor outages.** A round can still produce output if some miners are unavailable.

Outputs. For each miner j : a set of M bitstrings $\{x_1^{(j)}, \dots, x_M^{(j)}\}$, plus a signed timestamp proving they were returned within $t_{\text{threshold}}$, plus a checksum of the data.

Properties enforced.

- **Quantum-hardware-required.** On honest hardware, executing the circuit takes microseconds to milliseconds per shot. On classical simulation at the protocol’s (n, d) , it takes far longer than $t_{\text{threshold}}$ allows.
- **Not pre-computable.** The circuit is unknown until Leg 1 finalizes; miners cannot prepare samples in advance.
- **Subject to noise.** Real hardware has fidelity < 1 . Some fraction of shots are corrupted by noise rather than sampled from $p(x)$. This is what XEB will measure as ϕ_{honest} .

Open questions. How many miners per round (2? 5? variable?). How to handle miner failures and timeouts (drop just the late miner, or fail the round?). Whether the protocol mandates a specific quantum provider’s API or accepts any.

7. Leg 5 — Classical XEB verification

Role. Score the returned samples for evidence of genuine quantum sampling, and reject rounds that fail either the statistical test (XEB) or the timing test.

Inputs. The samples $\{x_1, \dots, x_M\}$ from each miner, the circuit C (recomputed deterministically from the public seed), and the timestamps.

Process. Validators perform three checks per round:

Check 1: Timing. Was the response received within $t_{\text{threshold}}$? Compare the timestamps. Cheap.

Check 2: Sample integrity. Sanity filter — no obviously fake patterns (all-zero strings, periodic data, etc.). Also cheap.

Check 3: XEB scoring. The expensive part. Validators select a random subset of m samples from the M submitted (Liu et al. used $m \approx 1500$). For each sample x_i in the subset, validators classically simulate the circuit using tensor-network contraction to compute the ideal amplitude:

$$p(x_i) = |\langle x_i | C | 0 \rangle|^2$$

Then compute the linear cross-entropy benchmark:

$$F_{\text{XEB}} = \frac{2^n}{m} \sum_{i=1}^m p(x_i) - 1$$

This score has the property that:

- Honest quantum samples with fidelity ϕ produce $F_{\text{XEB}} \approx \phi$.
- Uniform random samples produce $F_{\text{XEB}} \approx 0$.

The round is accepted iff $F_{\text{XEB}} \geq \chi$ AND the timing check passed. Otherwise rejected.

Outputs. Per round: an accept/reject decision, the computed F_{XEB} (preserved for audit and miner-reward calculation), the timing data, and the subset of indices used for verification.

The security argument. The Aaronson–Hung inequality bounds the classical adversary’s spoofing capability:

$$\phi_{\text{honest}} > \frac{A \cdot t_{\text{threshold}}}{B(n, d)}$$

where A is the adversary’s compute budget, $t_{\text{threshold}}$ is the timing window, and $B(n, d)$ is the cost of one classical circuit simulation. If this holds, no adversary can produce samples passing both XEB and timing checks. Parameter selection in Leg 2 must enforce this inequality.

Re-verifiability. Anyone with the seed (which is publicly committed), the parameter commitment (Leg 2), and the samples (published on-chain) can recompute C and re-run XEB to confirm the validator’s decision. This is the property Colton specifically called out: *validators or external auditors or anyone else* can verify the round.

Open questions. The computationally hardest aspect of the whole protocol. Specifically:

- What (n, d) are feasible for subnet validators? See the separate parameter feasibility study.
- What’s the optimal m that balances cost vs. statistical confidence?
- Should validators use the same tensor-network software and contraction order (canonical) or compete on contraction quality (adversarial)?
- How are multiple miners’ XEB scores aggregated when the protocol is multi-host?

8. Leg 6 — Randomness extraction

Role. Convert the raw biased quantum samples into uniformly random bits suitable for cryptographic use.

Inputs. The accepted samples (across one or more miners), an independent extractor seed (from drand or similar), and the certified min-entropy bound derived from the XEB score.

Process. Two sub-steps.

Min-entropy bounding. The raw samples have some min-entropy $H_{\min}$ that depends on the measured F_{XEB} . Approximately, $H_{\min} \approx M \cdot n \cdot f(F_{\text{XEB}})$ where f is a fidelity-to-entropy conversion derived from the Aaronson–Hung security analysis. The exact bound is technical; for our purposes, what matters is that we can compute a rigorous lower bound on the raw samples’ min-entropy after passing XEB.

Extraction. The raw samples plus the independent seed are fed into a strong randomness extractor. Two natural choices:

- **Toeplitz extractor (Liu et al. via Cryptomite library).** Fast, well-characterized, used by Liu et al. Output length depends on input min-entropy and chosen error.
- **TMPS Trevisan extractor (Kavuri et al.).** Slower but more output-efficient; widely used in DI-QRNG literature. Requires more seed bits.

The extractor takes biased input of min-entropy $H_{\min}$ and produces ℓ uniformly random output bits where ℓ is bounded by:

$$\ell \leq H_{\min} - 2\log(1/\varepsilon_x) - O(1)$$

for some extractor-specific constants and target error ε_x .

Outputs. A short string of near-uniformly-random bits with provable min-entropy $\geq \ell - \log(1/\varepsilon_x)$, plus the extractor seed used and the input min-entropy bound.

Why the independent seed is critical. A Trevisan extractor (or any seeded extractor) needs an independent uniform seed to extract from a non-uniform source. If the seed and the source share entropy with an adversary, the extraction is compromised. The independent seed should come from a source orthogonal to the VRF layer: drand is the obvious choice, since it’s:

- Cryptographically random (threshold-BLS signatures).
- Time-scheduled — drand publishes pulses on a known schedule, so the protocol can precommit to using “the next drand pulse after time T ” before that pulse is known.
- Independent of the subnet’s own randomness sources.

This matches Kavuri’s drand integration exactly.

Open questions. Which extractor (Toeplitz vs. TMPS Trevisan) is the right tradeoff for our throughput targets. Whether the independent seed comes from drand directly or via a wrapper chain. How many output bits per round (depends on M , n , F_{XEB} , ε_x).

9. Leg 7 — Twine-style hash-chain anchoring

Role. Create a tamper-evident audit trail spanning the entire protocol execution, so that every step can be cryptographically verified after the fact by anyone.

Inputs. All artifacts from Legs 1–6: VRF tuples (y_i, π_i) , parameter commitment, combined seed, circuit specification, per-miner samples and timestamps, validator XEB scores and decisions, drand pulse, extractor inputs and outputs, and the final extracted bits.

Process. Following the Kavuri Twine pattern, multiple independent parties maintain hash chains and *intertwine* them by including each other’s pulse hashes. The minimum set of chains for our subnet would be:

1. **VRF contributor chain(s)** — one per VRF participant, recording their published (y_i, π_i) tuples per round.
2. **Subnet orchestration chain** — analog of CURBy-Q. Records parameter commitments, the combined seed, the circuit specification, and the round-by-round protocol state.
3. **Quantum miner chain(s)** — one per miner. Records the sample data checksum and the timestamps.
4. **Validator chain** — records XEB scores, accept/reject decisions, and the verification subsets used.
5. **drand wrapper chain** — records the drand pulses consumed as extractor seeds.
6. **Output chain** — records the final extracted bits per round, with hashes pointing to the full audit trail.
7. **Time-stamping chain** (optional but recommended) — external RFC 3161 timestamp service to provide independent timing certification.

Each chain’s pulses include both internal links (to previous pulses on the same chain) and external mixins (hashes of pulses from other chains). Together they form a directed acyclic graph that is tamper-evident at the granularity of the entire protocol.

Outputs. A continuously updated DAG of hash-chained pulses, available via an HTTP API or directly on-chain. The final extracted bits are published on the output chain as the consumable subnet output.

Properties enforced.

- **End-to-end auditability.** Any external party can fetch the DAG, verify all hash links, and re-run every protocol step from public data.
- **Tamper-evidence.** Modifying any artifact retroactively requires rewriting all subsequent hash chains AND coordinating with every other party whose chain references the changed data. Detection is essentially guaranteed.
- **Time-ordering.** The DAG enforces a partial ordering on events: hypothesis before data, data before extraction, extraction before output. Adaptive attacks (where parameters are chosen after observing data) become detectable.

- **Decentralized trust.** No single party controls the audit trail. Even compromising the subnet operator wouldn't compromise an external auditor's ability to detect tampering.

Implementation. Following Kavuri's choices: SHA3-512 hashing, DAG-CBOR serialization, content-addressed pulses via CIDs (IPLD data model), RSA-4096 or ES256 signatures. The Twine specification and reference implementation at github.com/buff-beacon-project would be a starting point, adapted for the subnet's specific chain structure.

Open questions. Where the DAG is stored (IPFS? a dedicated indexer? on the host blockchain directly?). The economic model for chain operators (does the subnet pay for chain hosting, or do participants run their own?). The relationship between Twine and the Bittensor consensus mechanism — whether these are independent or integrated.

10. Summary of attacks the pipeline defends against

Attack	Description	Defended by
Biased seed	Adversary chooses a seed that produces favorable circuits	Leg 1 (VRF unbiasedness)
Grinding	Adversary tries many seeds to find favorable outputs	Leg 1 (VRF uniqueness per public input)
Last-revealer attack	In commit-reveal, last participant withholds to bias output	Leg 1 (VRF has no commit-reveal asymmetry)
Adaptive parameter tuning	Validator changes χ after seeing samples	Leg 2 (parameter precommitment)
Circuit ambiguity	Disagreement on which circuit was supposed to run	Leg 3 (deterministic expansion)
Pre-computation	Miner caches samples in advance	Legs 1+2+3 (seed unknown until just before round)
Classical spoofing	Adversary fakes samples without quantum hardware	Leg 4+5 (timing + XEB inequality $\phi > At/B$)
Heavy-output guessing	Partial classical simulation to pick high- $p(x)$ samples	Leg 5 (parameter choice in At/B)
Single-vendor compromise	One quantum provider's hardware is corrupted	Leg 4 (multi-host)
Forged samples	Miner submits unrelated bitstrings	Leg 5 (XEB scoring)
Late samples	Miner is slow	Leg 5 (timing check)
Non-uniform output	Raw quantum samples aren't directly usable as cryptographic randomness	Leg 6 (randomness extraction)
Compromised extractor seed	Extractor seed correlated with quantum samples	Leg 6 (independent drand seed)
Post-hoc tampering	Adversary alters historical records to fake past rounds	Leg 7 (Twine hash chains)
Collusion across audit parties	Multiple parties coordinate to rewrite history	Leg 7 (intertwined DAG makes this exponentially harder as network grows)

Every defense rests on a different assumption — cryptographic hardness for VRFs and hashing, computational hardness for XEB, physical correctness of the quantum hardware — which means a single broken assumption doesn't compromise the whole protocol.

11. What's still open

This pipeline is a working architecture, but several decisions and validations are still required:

Parameters. The (n, d) choice for the circuits is the load-bearing decision. The separate parameter feasibility study sketches the tradeoff between honest miner fidelity, classical spoofing resistance, and validator simulation cost. The provisional recommendation is $(n, d) = (32, 10)$ all-to-all, but actual benchmarking is needed.

VRF construction. Which VRF variant the team is standardizing on, and how it integrates with the existing Bittensor subnet infrastructure. Conversation with Medi needed.

Miner pool. Which quantum providers we target first (Quantinuum seems natural given Liu et al.’s hardware), how many miners per round, how to handle different platforms with different connectivities.

Multi-host integration details. How miners’ samples are combined — concatenated, cross-validated, threshold-aggregated. Affects both verification cost and security analysis.

Throughput vs. soundness tradeoff. Liu et al. used $\varepsilon = 2^{-64}$ and produced bits slowly. Kavuri achieved 512 bits per pulse at the same ε . What’s the right target for an on-chain randomness service consumed by casinos, NFTs, and loot boxes? Likely depends on go-to-market timeline.

On-chain integration. How the output chain interfaces with consumers, gas costs for verifying VRF proofs and hash chains on-chain, whether the Twine DAG lives on the host blockchain or as an off-chain attested data structure.

Economic model. Miner reward formula, validator reward formula, treatment of failed rounds, slashing conditions for misbehavior.

These are not blockers to building the architecture — they are decisions to be made along the way, with the seven-leg structure providing the framework within which each is resolved.

12. Cross-references to companion documents

- Liu et al. walkthrough — detailed reading notes on the RCS-XEB certification mechanism.
- Kavuri et al. walkthrough — detailed reading notes on the Twine traceability protocol.
- Liu vs. Kavuri comparison — side-by-side analysis of the two paradigms and what’s transferable.
- Parameter feasibility study — back-of-envelope analysis of the (n, d) tradeoff and recommended starting parameters.

Together these four documents plus this pipeline spec form the working understanding of the subnet architecture. Anything not addressed here is in one of the others; anything not in any of them is genuinely open.